Кейс «Лаборатория протоколов» · Мадригал

# protoledger

Журнал исследования протокола: от записей трафика к проверяемому, повторно применимому и переносимому описанию обмена.

локально и офлайн

окно · браузер · командная строка

Rust + Nuxt

github.com/protoledger/protoledger

# Проблема

Подключить систему к устройству без документации на протокол — обычная задача интеграции: контроллеры, измерительные приборы, IoT, устаревшие сервисы.

Сегодня это Wireshark, hex-редактор, заметки и одноразовые скрипты.

### Что ломается

- Выводы **не привязаны к байтам** конкретной записи
- Догадка выглядит так же, как установленный факт
- На новой записи всё проверяется **заново руками**
- Коллеге передаётся результат, но **не основания**
- Дефекты захвата (потери, повторы, offload) путают выводы

# Решение: цикл исследования в одном инструменте

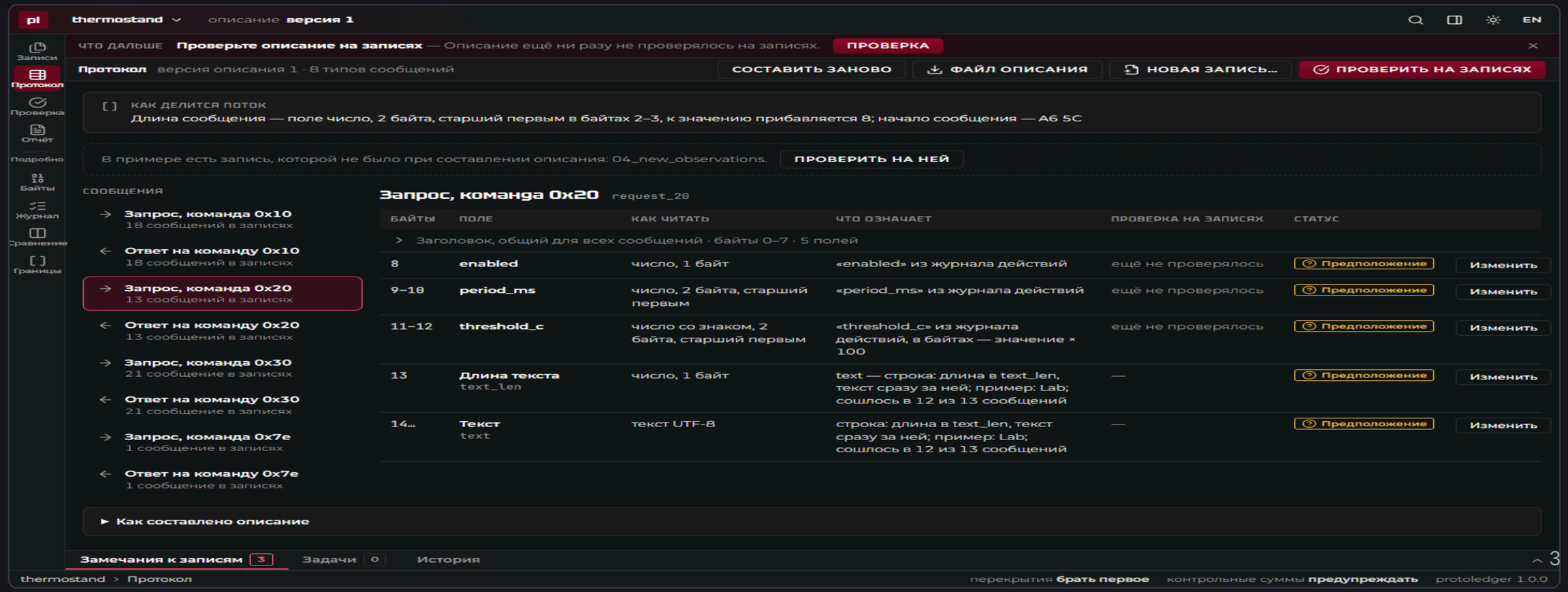
**Записи**
сборка TCP, дефекты явно

**Исследование**
границы, обмены, сравнение

**Описание**
установлено · предположение · не описано

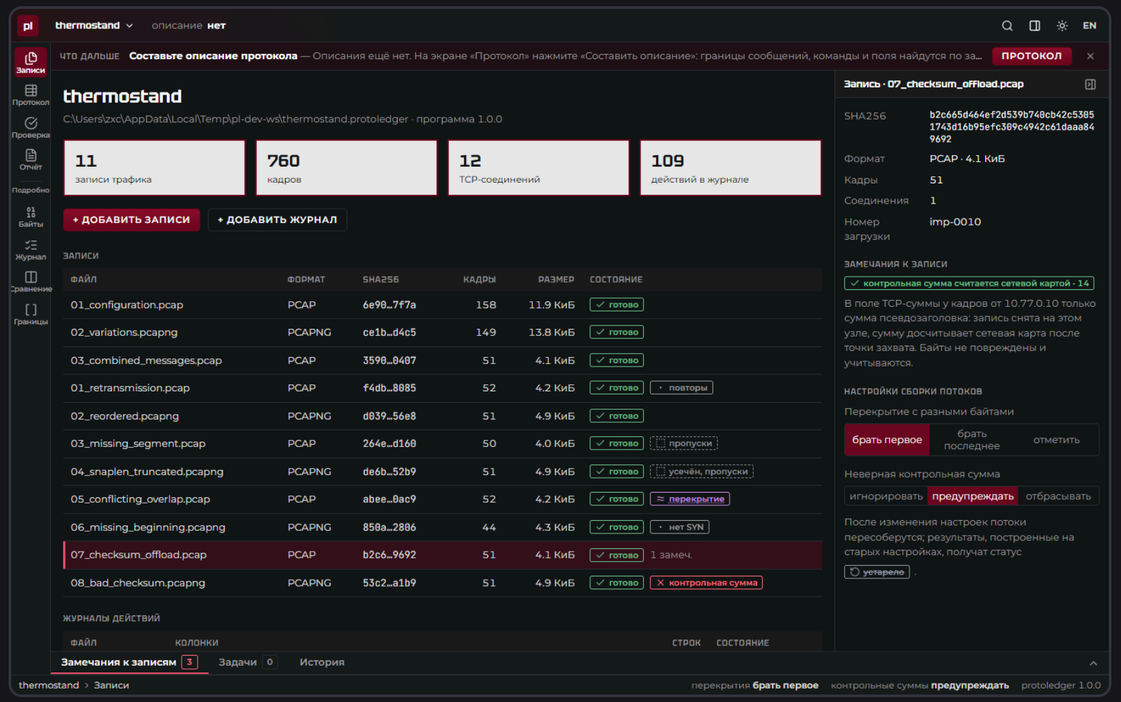
**Проверка**
все записи, расхождения

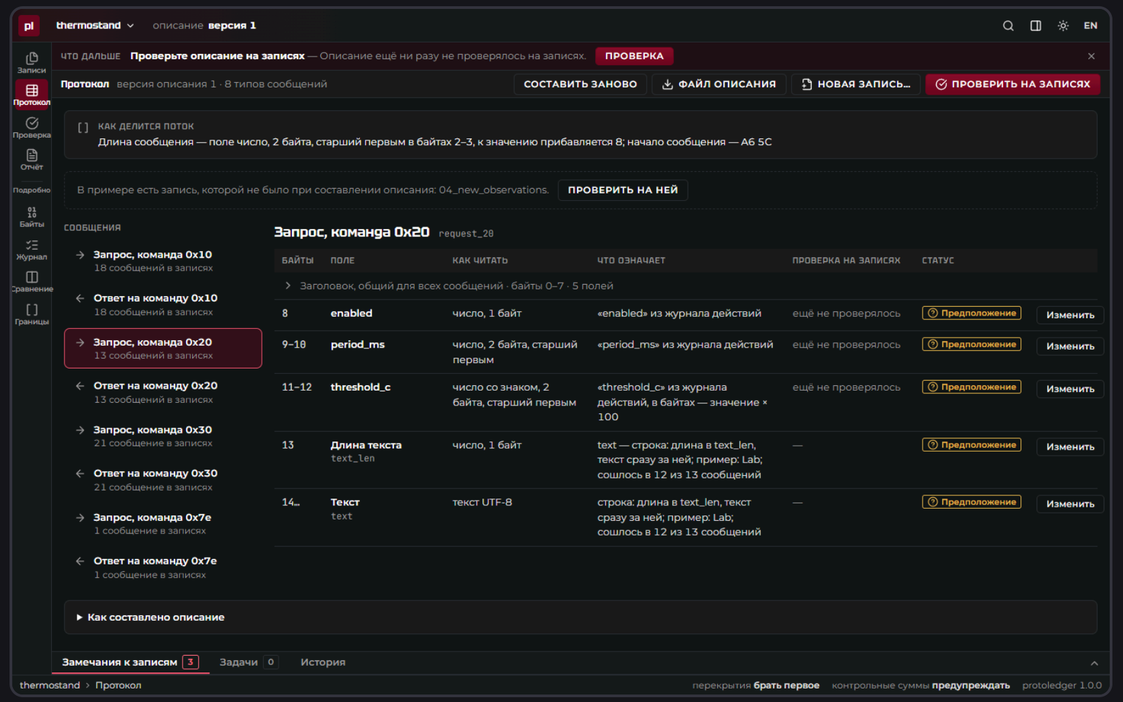
**Передача**
отчёт, YAML, папка проекта



# Демо 1. Записи → описание протокола одной кнопкой

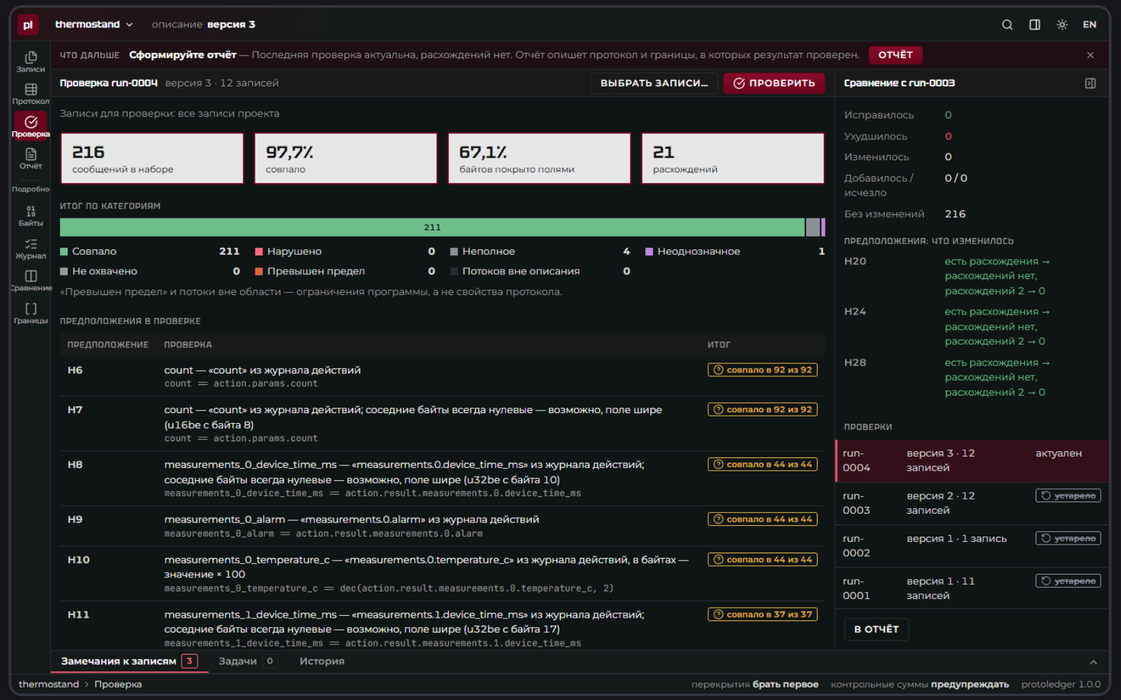
11 записей «Термостенда» и журнал из 109 действий; колонки журнала и границы сообщений программа находит сама

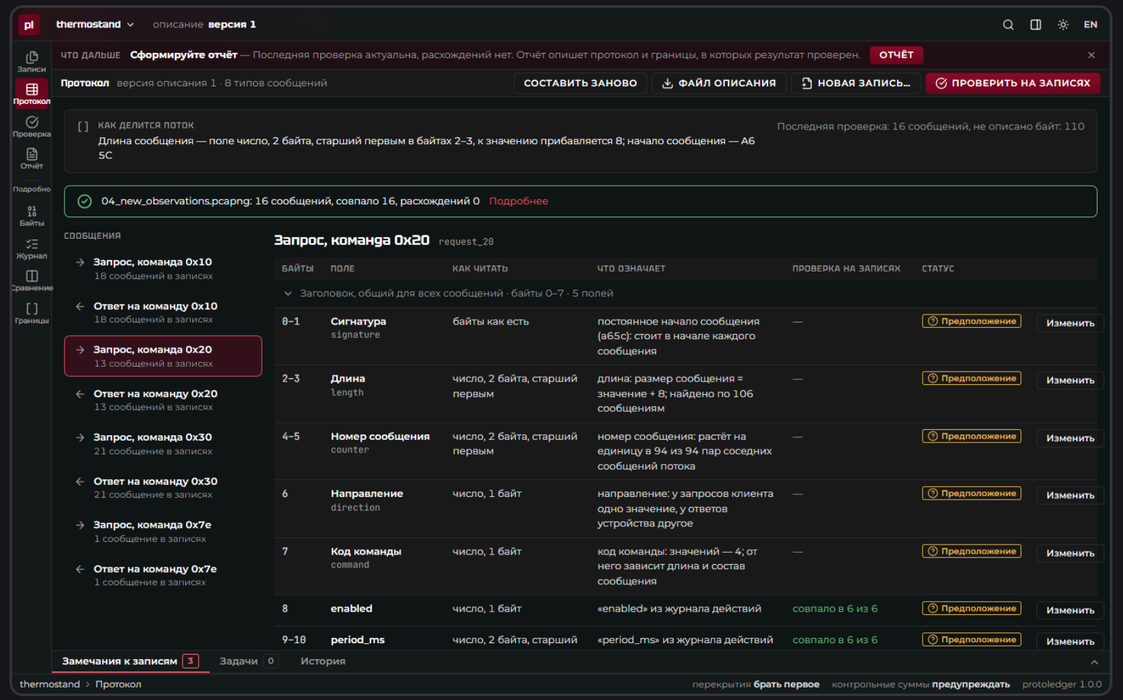



Повторы, пропуски, обрезанные кадры и offload видны явно. «Составить описание»: сигнатура A6 5C, длина в байтах 2–3, код команды в байте 7, поля связаны с параметрами журнала: порог 19 °C → 07 6C = 1900.

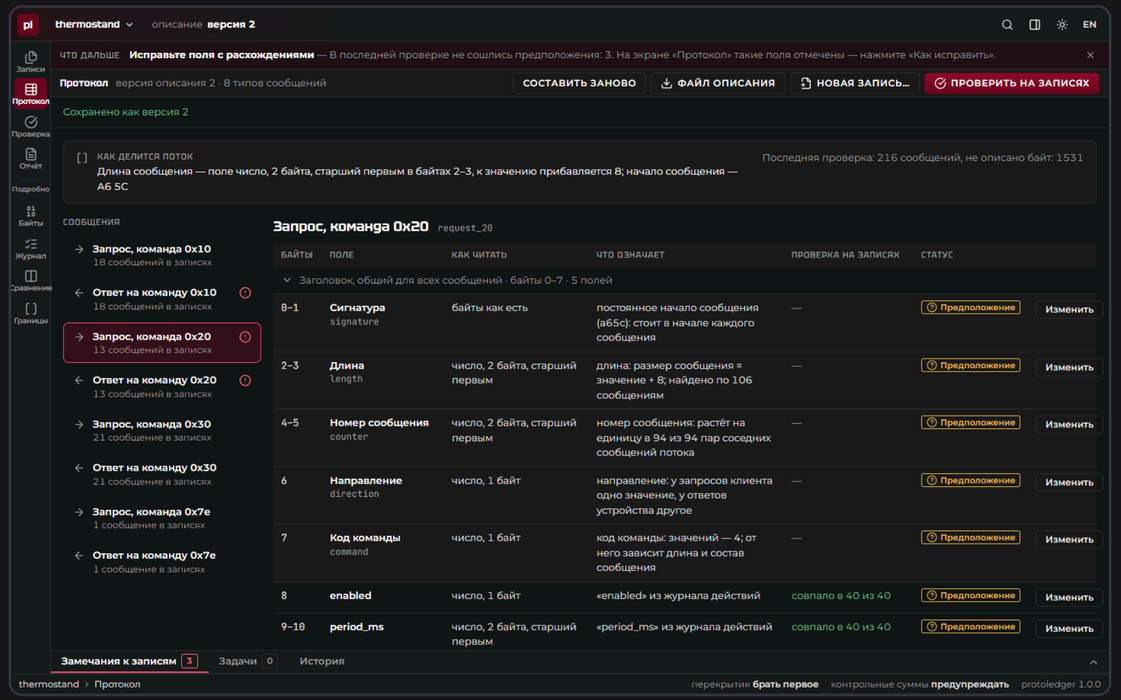
# Демо 2. Проверка на записях и на новой записи



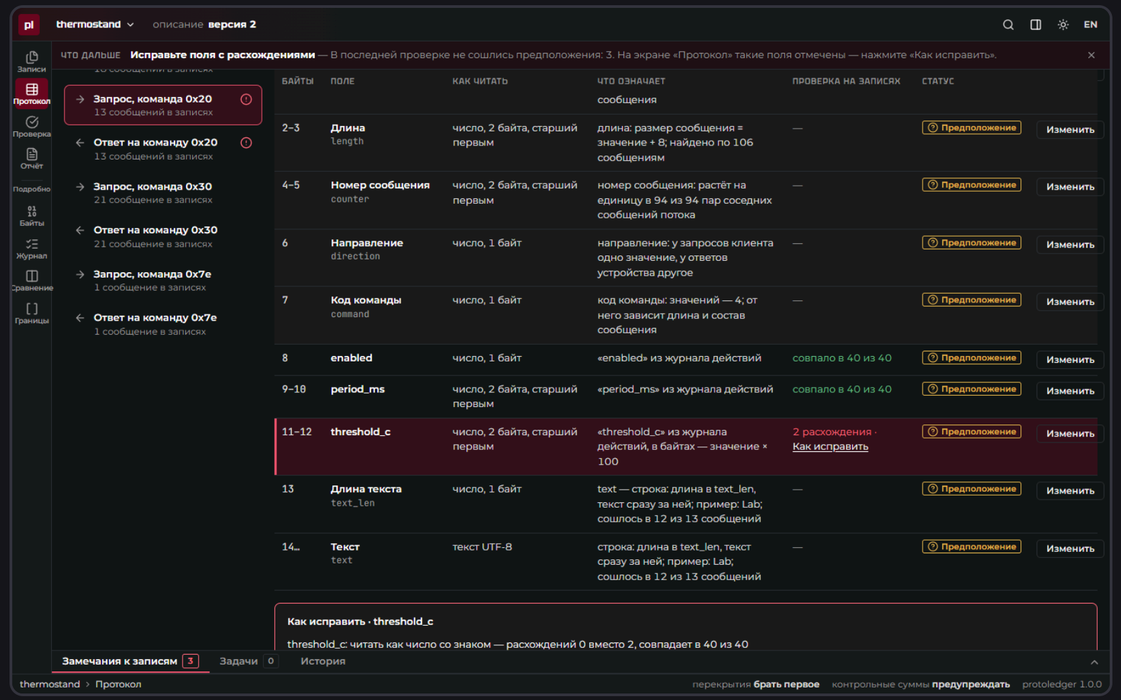




Описание применяется ко всем записям: 200 сообщений, у полей — «совпало в 40 из 40». Запись, которой не было при составлении описания, разбирается без правки: 16 из 16.

# Демо 3. Ошибка → расхождение → уточнение




Порог прочитан как число **без знака**: на пороге −2,5 °C байты **FF 06** дают 65 286 вместо −250 — 2 расхождения с конкретными сообщениями.




«Как исправить»: читать как число **со знаком** — расхождений 0 вместо 2. Новая версия описания, проверка заново: совпало в 40 из 40.

# Как это работает

```
SPA (Nuxt) — в браузере или окне Tauri
        |  HTTP JSON /api · SSE · только 127.0.0.1
        |  Host/Origin + токен сессии
движок protoledger (Rust, один бинарник)
  pl-server    API, защита, встроенная SPA
  pl-app       сценарии, реестр задач
  pl-capture   pcap/pcapng → индекс кадров
  pl-reassembly TCP: потоки, дыры, перекрытия
  pl-actions   журнал действий (CSV)
  pl-interp    описание протокола, выражения
  pl-verify    проверки, расхождения
  pl-report    отчёт MD / HTML
  pl-project   папка проекта, атомарная запись
        |
<имя>.protoledger/  YAML/JSON + копии записей
```

## Один движок — три оболочки

Окно приложения (.msi, .deb, AppImage), браузер (`docker compose up` или бинарник `serve`), командная строка: `apply`, `verify`, `report`, `draft`.

## Без базы данных

Состояние — папка проекта: текстовые YAML/JSON и неизменные копии записей с sha256. Перенос — копирование папки, `protoledger verify` воспроизводит проверки.

# Достоверность и воспроизводимость

### Знание со статусом

- Поле — установлено, предположение или не описано
- «Совпало в k из n», а не «доказано»
- Каждый вывод — якорь на байты записи

### Честные категории

- Свойства протокола: нарушено, не охвачено
- Дефекты записи: неполное, неоднозначное
- Ограничения реализации: превышен предел, поток вне области

### Воспроизводимо

- Все версии описания хранятся
- Проверка подписана хешами входов: устаревший виден
- Детерминированный вывод: CLI ≡ интерфейс

### Безопасность

- Только 127.0.0.1, проверка Host и Origin, токен сессии, CSP
- Без `unsafe`, без `unwrap` на данных

### Злые входы

- 93 «злых» файла в наборе, 5 целей фаззинга
- Битый кадр — диагностика, а не падение

### Проверки в CI

- 419 тестов движка, fmt и clippy без предупреждений, аудит лицензий
- 16 эталонов, демо по кнопкам в браузере

# Результаты на данных кейса

| «ТЕРМОСТЕНД», НАБОР ОРГАНИЗАТОРОВ | |
|---|---|
| Записей / действий журнала | **11 / 109** |
| Сообщений после уточнения, расхождений | **216 / 0** |
| Дополнительная запись без правки описания | **16 из 16** |
| Черновик описания без знаний о протоколе | **16 из 16** |
| Диагностические записи: дефект распознан | **8 из 8** |
| Modbus и MQTT — тот же движок, без кода о них | **эталоны в CI** |
| Чужая запись 4SICS (Modbus/TCP) | **19 из 19** |

| ПРОФИЛЬ Т3: 100 МИБ, 250 ТЫС. КАДРОВ, 1000 СОЕДИНЕНИЙ, ОТВЕТЫ ДО 64 КИБ; 4 VCPU / 8 ГИБ | |
|---|---|
| Разбор записи (CLI) | **0,20 с** |
| Импорт в проект: копия, sha256, сборка | **0,80 с** |
| Проверка 235 тыс. сообщений с предположениями | **0,78 с** |
| Отчёт | **0,02 с** |
| Пик памяти движка | **200 МиБ** |

# Ограничения и развитие

### Сейчас — честно

- Область Т3: Ethernet + IPv4 + TCP; IPv6, UDP и фрагменты IP — пропускаются с диагностикой
- Шифрованный трафик не разбирается — показывается как неописанные байты
- Запись читается в память целиком (предел 1 ГиБ)
- Предположение не становится доказанным: проверено только на наблюдавшихся значениях
- Установщики — Windows и Linux

### Дальше

- IPv6 и UDP — новые слои того же конвейера
- Чтение записи частями для файлов больше памяти
- Эксперименты со стендом из приложения: конструктор сообщений
- Двойник устройства по выученным парам «запрос → ответ»
- Экспорт парсера: Wireshark уже есть, далее Kaitai Struct

# Команда и ссылки

| | |
|---|---|
| sleepyhead-dev | Backend, DevOps |
| OddS-Programming | Backend |
| rinna32 | Frontend |
| sky1768205 | Frontend |

**Репозиторий:**
github.com/protoledger/protoledger

**Релиз v1.0:** установщики Windows и Linux, бинарники — страница Releases

**Запуск:** `git clone --recursive` → `docker compose up --build` → localhost:8080

**Как пользоваться:** docs/guide.md · **проверка ТЗ:** docs/requirements.md